UKSHATI TECHNOLOGIES PVT LTD

CRM Module and Documentation

Name: Poornapragna GB

Sahyadri college of engineering and management

Duration: 31 Jan – 5 Feb 2026

CRM Module Overview:

The Customer Relationship Management (CRM) module is designed to manage and organize customer-related information. It helps to maintain records of customers, projects, reminders. The main objective of this module is to Manage client relationship and interactions, track important tasks and deadlines, and monitor ongoing and upcoming initiatives. The customer card in this module provides functionalities such as adding and managing customer details. Remainder management card provides functionalities such as scheduling reminders, and sending notifications to users. Project management card provides functionalities such as adding and managing the projects.

1. **home.js**

   Purpose: The home.js file acts as the main dashboard of the CRM module. It provides a centralized overview of customers, projects, and reminders. This page helps users quickly understand the current status of activities without navigating to multiple pages.

   Functionalities Implemented:
   • Displays total number of customers
   • Shows active projects count
   • Displays pending tasks
   • Shows completed projects
   • Provides quick navigation cards for Customers, Reminders, and Projects
   • Displays recent projects in tabular format
   • Shows upcoming reminders
   • Includes search functionality for customers
   • Displays graphical charts for project growth and customer distribution

   Data is fetched from:
   - ✓ **/api/customers** → customer details
   - ✓ **/api/tasks** → project/task data
   - ✓ **/api/reminders** → reminder information

   These APIs are called inside **useEffect()** when the page loads.
   useEffect() and useState() are the React hooks

   useState() → manage dashboard states (customers, projects, reminders, loading, metrics)

   use client→ Tells **Next.js** that this page must run on the **client side (browser)**

   import { FiUsers, FiBell, FiPackage, FiPieChart, FiSearch } from "react-icons/fi";

   - FiUsers → Customers
   - FiBell → Reminders / Notifications
   - FiPackage → Projects
   - FiPieChart → Analytics/Reports
   - FiSearch → Search bar

import { motion } from "framer-motion"; → Adds smooth animations.

import Link from "next/link"; → Used for **client-side navigation** and it is faster than <a> tag.

import dynamic from "next/dynamic"; → Loads heavy components **only in browser.**

import BackButton from "@/components/BackButton"; →navigation

import Tilt from 'react-parallax-tilt'; → 3D hover effect.

import { CardSkeleton, TableSkeleton } from "@/components/skeleton"; → loading placeholder

ssr:false → avoids server-side crash.

const metricsData = [→ Each object represents one dashboard card.

const CRMDashboard = () => { → This is the main CRM dashboard component**.**

const [notification, setNotification] = useState(null); → Stores alert messages

const [isLoading, setIsLoading] = useState(true); → Shows loaders until API data loads

const [customers, setCustomers] = useState([]);

const [projects, setProjects] = useState([]); Stores fetched backend data

const [reminders, setReminders] = useState([]);

const [metrics, setMetrics] = useState(metricsData); → Stores dashboard card values

const [searchTerm, setSearchTerm] = useState("");

const [filteredCustomers, setFilteredCustomers] = useState([]); → Used for customer search

const [isMounted, setIsMounted] = useState(false); → Prevents mismatch (Next.js issue)

useEffect(() => {

setIsMounted(true); → Ensures page renders only after browser loads

}, []); Prevents SSR + client mismatch

useEffect(() => { → Fetching CRM data

const fetchData = async () => { This runs after component mounts

setCustomers(customersData.customers || []); Saving data into state

setProjects(Array.isArray(projectsData) ? projectsData : []); →Prevents crashes if API

setReminders(Array.isArray(remindersData) ? remindersData : []); response is empty

updateMetrics(customers, projects, reminders); → Calculates: Total customers Active projects,

Pending reminders,Completed projects

const showNotification = (type, message) => { → Show messages and Auto-hides after 3 seconds

metrics.map((metric) => (→ Loops through metrics array then Displays cards

<CRMDashboardVisualizations /> → Loads charts dynamically Prevents SSR crash

export default CRMDashboard; → Makes page accessible at /crm/home

2. **customers.js:**

Purpose: The customers.js file main purpose is to manage all customer-related information in a centralized way. This module acts as the **core database interface** for handling customer records. It is responsible for handling all customer-related operations such as creating, viewing, editing, deleting, importing, and exporting customer data. This page serves as the main interface between the **frontend user interface** and the **backend customer APIs**.

Functionalities Implemented:

- Add Customer
- Edit Customer
- Delete Customer
- CSV Export

import Papa from "papaparse"; → Used for CSV Import/Export**.**

import BackButton from "@/components/BackButton"; → Reusable component to go back to CRM dashboard.

import ScrollToTopButton from "@/components/scrollup"; → Floating button to scroll page.

import { TableSkeleton, FormSkeleton } from "@/components/skeleton"; → Loading placeholders such as tables & forms

import * as XLSX from "xlsx"; → Used for Excel Import.

export default function Customers() {→ Represents Customer Management Page.

const [customers, setCustomers] = useState([]); → Stores all customers list from API.

const [name, setName] = useState(""); → Stores customer name input.

const [phone, setPhone] = useState(""); → Stores phone number.

const [altPhone, setAltPhone] = useState(""); → Stores remarks/notes

const [status, setStatus] = useState("lead"); → Stores status

const [editingRemarks, setEditingRemarks] = useState({}); → Tracks which remark is being edited inline.

const [csvFile, setCsvFile] = useState(null); → Selected import file.

const [loading, setLoading] = useState(true); → Loading indicator.

const [error, setError] = useState(null); → Stores API errors.

const [searchTerm, setSearchTerm] = useState(""); → Search input value.

const [importing, setImporting] = useState(false); →True while importing file

const [importProgress, setImportProgress] = useState(""); →Shows progress message during import

const res = await fetch("/api/customers"); → Calls backend API.

setCustomers(data.customers); → Saves customers into state.

3. **reminders.js**

Purpose: The Reminders module is used to schedule and manage customer-based reminders inside the CRM system. It helps users create time-based notifications for follow-ups, tasks. This module allows users to select a customer, enter a reminder message, choose date and time, and receive notifications at the scheduled time. Reminder is linked to specific customer (via customer ID) Helps track customer-specific follow-ups.

Functionalities Implemented:

- User selects a customer from dropdown
- Enters reminder message
- Selects date
- Selects time
- Clicks Schedule Reminder
- Reminder gets stored in database
- Supports future reminders
- Shows Notifications Enabled status
- Displays Upcoming reminders, Past reminders, All reminders

import Swal from 'sweetalert2'; → For popup alerts.

import { FiAlertCircle, FiUser, FiClock, FiCalendar, FiMessageSquare, FiTrash2, FiPlus, FiBell } from 'react-icons/fi'; →

- FiBell → notifications
- FiUser → customer
- FiClock → time
- FiCalendar → date
- FiTrash2 → delete
- FiPlus → add reminder
- FiMessageSquare → message
- FiAlertCircle → alerts

const [reminders, setReminders] = useState([]); → Stores reminders list.

const [customers, setCustomers] = useState([]); → Stores customers list for dropdown.

const [loading, setLoading] = useState(true); → Shows loading skeleton.

const [notificationsEnabled, setNotificationsEnabled] = useState(false); → Tracks notification permission.

const [formData, setFormData] = useState(→ Stores form inputs.

{ message:'', date:'', time:'', customerId:'' });

const [activeTab, setActiveTab] = useState('upcoming'); → Tracks selected filter tab.

const filteredReminders = useMemo(() => { ... }, [reminders, activeTab]); → filtering remainders

requestNotificationPermission() → Requests permission Enables notifications.

setupNotificationListener() → Listens for service worker messages.

fetch('/api/reminders', { method:'POST' }) → Sends reminder data Saves to database

deleteReminder(rid) → Delete API call and removes reminder

4. **Project.js**

Purpose: The Project Management is designed to manage all customer-related projects in the CRM. It allows users to create new projects, assign them to customers, set timelines, update statuses, edit project details, and delete projects. This module helps in tracking the progress ongoing, completed, or on-hold projects.

Functionalities Implemented:
• Add new project
• Assign project to a customer
• Set start date and optional end date
• Select project status
• View all projects in table format
• Search projects by name
• Edit project details
• Delete project
• Auto date formatting

const [projects, setProjects] = useState([]); → Stores list of all projects.

const [customers, setCustomers] = useState([]); → Stores customers for dropdown.

const fetchProjects = async () => {→ Creates async function to get project data.

const res = await fetch("/api/tasks"); → Calls backend API for projects.

if (!res.ok) throw new Error("Failed to fetch projects"); → Handles API failure.

const data = await res.json(); → Converts response to JSON.

await new Promise(resolve => setTimeout(resolve, 300)); → Adds small delay for smoother loading effect.

setProjects(data); → Stores projects in state.

const formatDate = (dateString) => {→ Formats dates properly.

const statusOptions = [...] → Defines dropdown values: ongoing, hold, completed.

const projectData = { ... } → Creates project object (pname,cid,date,status) to send to backend.

fetch(url, { method, headers, body }) → Sends data to server.

setEditId(project.pid); → Marks editing mode Then fills form with existing values.

fetch(`/api/tasks?pid=${pid}`, { method: "DELETE" }) → Ensures safe deletion.

const filteredProjects = projects.filter(...) → Filters projects by name using searchTerm.

5. **test-notification.js**

Purpose: This page is a testing utility page used to Check browser notification support. View current notification permission status. Request notification permission manually Initialize the notification system. It is mainly used for debugging and testing reminders/alerts in the CRM

Functionalities Implemented:

- Check permission status
- Button allows user to enable notifications.
- Background notification listener
- Service worker
- Safe browser execution

import { initializeNotifications } from '@/utils/notification-system'; →Imports a helper function that: registers service worker, sets up background checks, prepares notifications

const [permissionStatus, setPermissionStatus] = useState('unknown'); →Stores notification

if (typeof window === 'undefined') return; →Prevents server-side execution. Because Notifications only work in browser.

setPermissionStatus(Notification.permission); → Reads current browser permission.

const requestPermission = async () => { → Called when button clicked.

const permission = await Notification.requestPermission(); → Shows popup notification.

setPermissionStatus(permission); → Updates UI instantly.

6. CRMdash.js

Purpose: To display CRM analytics using charts**.** Project Growth (Line chart), Customer Distribution (Pie chart). Helps users to understand project trends, view customer types, analyze system data visually, quick business overview

Functionalities Implemented :

• Fetches project data from /api/tasks
• Displays monthly growth using line chart
• Fetches customers from /api/customers
• Shows customer distribution using pie chart
• Fetches reminders from /api/reminders
• Updates charts dynamically
• Responsive design for different screens.

import { Line, Pie } from 'react-chartjs-2'; → Import chart components for Line chart Pie chart

import { Chart as ChartJS, CategoryScale, LinearScale, PointElement, LineElement, BarElement, ArcElement, Tooltip, Legend } from 'chart.js'; → Import chart.js core modules.

const demoData = { ... } → Used when API has no real data

const orderedMonths = ['Jan','Feb',...] → Used for X-axis labels of line chart.

const [customerData, setCustomerData] → Stores customer type counts for pie chart.

const [reminderData, setReminderData] → Stores reminder priority counts.

** ** → Two charts side-by-side.

<Line data={lineChartData} /> → Shows project growth.

<Pie data={pieChartData} /> → Shows customer distribution.